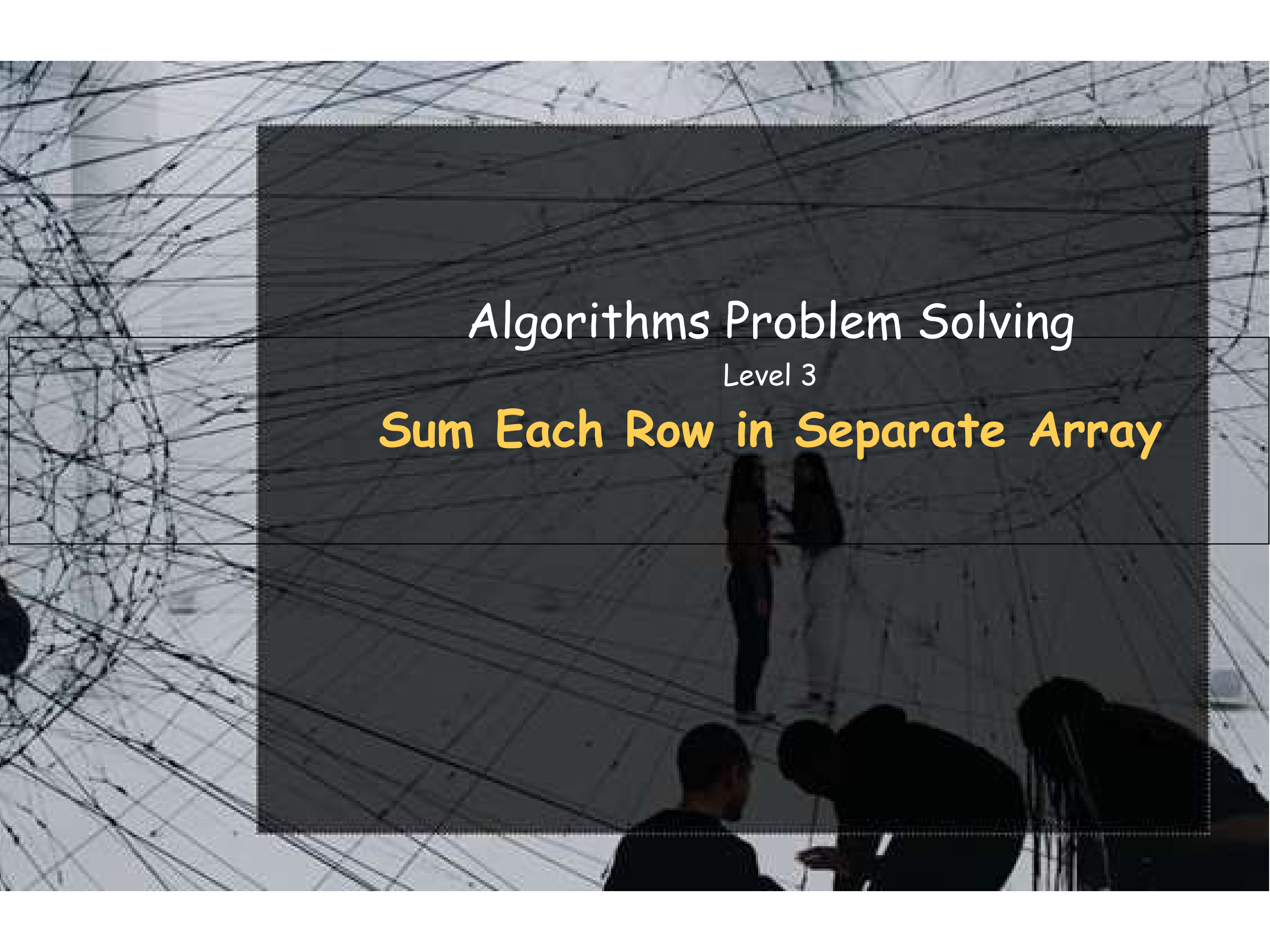


Algorithms Problem Solving

Level 3

Sum Each Row in Separate Array



Algorithms Problem Solving

Level 3

Sum Each Row in Separate Array

Definition

Sum Each Row in Separate Array

-  The program creates a 3x3 matrix filled with random numbers.
-  It calculates the sum of each row and stores the results in a separate vector called `rowSums`.
-  The original matrix and the row sums are then printed.

Explanation

`rowSums`

-  Each row has its own element in the `rowSums` vector.

-  The sum of row `i` is stored in `rowSums[i]`.

Separating Calculation and Printing

-  The calculation of the row sums is separated from the printing operation.

-  This makes the code more organized and allows the calculated sums to be reused later.

`RowSums()`

-  Calculates the sum of every row and returns a separate vector containing the results.

Procedures

❑ Create a 3x3 matrix using:

```
vector<vector<int>> matrix(3, vector<int>(3));
```

❑ Fill the matrix with random numbers between 1 and 100 using:

```
FillMatrixWithRandomNumber(matrix, 3, 3);
```

❑ Print the original matrix using:

```
PrintMatrix(matrix, 3, 3);
```

❑ Calculate the sum of each row inside `RowSums()` and store the results in a new vector:

```
vector<int> rowSums = RowSums(matrix, 3, 3);
```

❑ Print the row sums using:

```
PrintRowSums(rowSums);
```

▣ `FillMatrixWithRandomNumber()`

▨ Generates random numbers and fills the matrix.

```
matrix[i][j] = rand() % 100 + 1;
```

▣ `PrintMatrix()`

▨ Prints the matrix in a formatted way using `setw()` from `*iomanip*`.

Procedures

`RowSums()`

 Calculates the sum of each row:

```
for (int i = 0; i < rows; i++)  
{  
    for (int j = 0; j < columns; j++)  
    {  
        rowSums[i] += matrix[i][j];  
    }  
}
```

 Then returns the `rowSums` vector.

`PrintRowSums()`

 Prints the sum stored for each row in the `rowSums` vector.

💡 Example

The following is a 3x3 random matrix:

53 43 6

65 83 48

64 30 36

The following are the sum of each row in the matrix:

Row 1 Sum = 102

Row 2 Sum = 196

Row 3 Sum = 130

➡ Result

The `rowSums` vector contains:

102 196 130

Note

- ◆ ``srand(time(0))`` initializes the random number generator so different runs normally produce different sequences.
- ◆ The ``rowSums`` vector should contain one element for each row in the original matrix.
- ◆ A ``vector`` provides flexible dynamic storage and can grow when needed.

⊘ Warning

- ◆ Make sure `rowSums` contains enough elements before using `rowSums[i]`.
- ◆ If `rowSums` is empty, accessing `rowSums[i]` is invalid.
- ◆ When creating the sums vector, initialize it with the number of rows, for example:
`vector<int> rowSums(rows, 0);`



Comparison

▨ Problem #02 - Sum Each Row

- ✓ Calculates the sum of each row.
- ✓ Uses `rowSum` temporarily.
- ✓ Prints each result directly.
- ✓ No separate storage for all row sums.

▤ Problem #03 - Sum Each Row in Separate Array

- ✓ Calculates the sum of each row.
- ✓ Stores the results in `rowSums`.
- ✓ Prints the stored results afterward.
- ✓ The sums can be reused later for additional calculations.

✦ Summary

- ✓ Calculate the sum of each row in a 2D matrix.
- ✓ Store each row's sum in a separate `rowSums` vector.
- ✓ `rowSums[i]` corresponds to row `i`.
- ✓ Separate calculation from printing to improve code organization.
- ✓ Storing the results allows them to be reused later.

Thank you 😊